Urumi AI FDE Internship - Round 1: MCP Gateway on Kubernetes

Hiring process

- **Round 1:** MCP Gateway on Kubernetes (this assignment)
- **Round 2:** Interview and live walkthrough of your submission
- **Round 3:** Interview with founders

Role context

This internship is AI-focused, and Kubernetes is a core part of the daily work. This assignment tests both: you deploy a real stateful application on Kubernetes, then build the AI tooling layer on top of it.

You have two weeks. The task is difficult, and you will learn a lot. A working core is worth more than an unfinished bonus.

AI usage

You are encouraged to use AI tools to move faster. However:

- You must be able to explain what you built and why it works.
- You should understand the internals: the Kubernetes resources, the MCP protocol flow, how credentials move, and where isolation is enforced.
- Do not submit AI-generated work you do not understand. In the demo video and the interview we will ask you to walk through the solution end to end and defend your choices.

Disclaimer

The work you do for this task is yours and you own the copyright. Urumi will not use the test task code in production.

Problem statement - WooCommerce hosting with Team MCP Gateway

Build WooCommerce hosting platform with a team MCP gateway on Kubernetes, with a WooCommerce store as its first data source. An admin connects MCP servers once in a dashboard, and every team member can then use all of them through a single MCP endpoint and through a chat agent in the dashboard.

The whole system must run on local Kubernetes and be deployable to a production-like VPS (k3s is fine) using the same Helm charts, with configuration changes only.

The system has four layers:

1. **Store.** One WooCommerce store (WordPress and WooCommerce) running on Kubernetes.
2. **WooCommerce MCP server.** An MCP server you write yourself that exposes the store's products and orders as tools.
3. **Gateway and dashboard.** A React dashboard and backend where admins register MCP servers. The backend is an MCP client to every registered server and is itself an MCP server that exposes all their tools through one endpoint.
4. **Chat agent.** A basic chat in the dashboard that answers questions by calling tools through the gateway.

Both the chat agent and external clients reach every MCP server through the gateway; nothing else talks to the upstream servers directly.

User stories

As an **admin**, I should be able to:

1. Open the dashboard and see the registered MCP servers with their status (Connected / Unreachable / Disabled) and the tools each one exposes.
2. See the WooCommerce MCP server already registered as the first server.
3. Add another MCP server by URL and credential, and edit, disable or remove it.
4. Invite team members and issue or revoke their API keys.
5. See an audit log of every tool call: who, which server and tool, when, how long, and whether it succeeded.

As a **team member**, I should be able to:

1. Generate my own API key in the dashboard.
2. Add one gateway URL and my key to an MCP client such as Claude Code or Cursor, and see the tools from every enabled server without configuring any of them myself.
3. Use the chat agent in the dashboard to ask questions that are answered with those same tools.
4. Never see or need the credentials of the upstream servers.

Definition of done (must be testable)

Your demo must show this flow working end to end, with two different users.

Store

- Open the storefront, add a product to the cart and check out with a test-friendly method (COD or a dummy gateway is fine).
- Confirm the order is visible in WooCommerce admin.

Chat agent, as user A

- Ask the agent to create a product with a name and price. The product appears on the storefront.
- Place an order for it on the storefront by hand, then ask the agent what orders came in today. It returns that order.

External MCP client, as user B

- Connect Claude Code, Cursor or MCP Inspector to the gateway using only the gateway URL and user B's API key.
- See the tools of every enabled server, and ask the same orders question successfully.
- Run one prompt that needs tools from two different upstream servers.

Team and control

- Revoke user B's key. Their next call is rejected.
- Disable one upstream server. Its tools disappear from both the chat agent and the external client.
- Kill the pod of one upstream server. The other servers keep working, and the dashboard shows the dead one as unreachable.
- Every call above appears in the audit log, attributed to the right user.

Requirements (mandatory)

Kubernetes and Helm

- Must run on local Kubernetes (Kind, k3d or Minikube).
- Must be deployable to a production-like VPS (k3s is fine) using the same Helm charts.
- Helm is mandatory (no Kustomize). Local and production differences are handled through Helm values, for example `values-local.yaml` and `values-prod.yaml`.
- Everything runs in the cluster: the store, its database, the WooCommerce MCP server, the gateway backend and the dashboard.
- The store's database and WordPress content use persistent storage.
- The storefront and the dashboard are exposed through Ingress with stable URLs. Document your local domain approach.
- Readiness and liveness checks on every workload, and resource requests and limits.
- No hardcoded secrets in source code. Credentials live in Kubernetes Secrets.

WooCommerce MCP server

- You must write it yourself. Do not install an existing WooCommerce MCP server or plugin.
- It speaks MCP over Streamable HTTP.
- It exposes at least these tools: list products, create product, list orders, get order, update order status.
- It is not reachable from outside the cluster. Only the gateway can call it.
- Document how it talks to the store (REST API, WP-CLI or another way) and why you chose that.

Gateway

- Admins can add, edit, disable and remove upstream MCP servers that speak Streamable HTTP, each with an optional static credential (header or bearer token).
- The gateway exposes one MCP endpoint. Listing tools returns the tools of every enabled server, and calling a tool is routed to the right server.
- Two servers may expose a tool with the same name. Handle this with a scheme you document.
- A slow or dead upstream must not break tool listing or calls to other servers. Calls have timeouts and return a clear error.
- Upstream credentials stay on the server side. They are never sent to the browser or to an MCP client.
- You must connect at least three upstream servers: your WooCommerce server, and two others of your choice.

Team and access

- At least two roles: admin (manages servers and members) and member (uses tools).
- Each user has their own API key for the gateway. Keys can be revoked, and are stored hashed.
- Every tool call is written to an audit log with user, server, tool, timestamp, duration and outcome, and the log is visible in the dashboard.

Chat agent

- A chat in the dashboard that uses an LLM to call tools through the gateway and shows which tools it called.
- It acts as the logged-in user, so its calls appear in the audit log under that user.
- The model, provider and design are your choice. A free tier or a local model is fine. We grade that the tool loop works, not the quality of the model's answers.
- LLM API keys follow the same rule as every other secret.

Bonus: multi-store provisioning

This section is optional. Attempt it only after the core works end to end.

Turn the single store into a store provisioning platform. From the dashboard, a user can create and delete stores, and each new store is connected to the gateway automatically.

- **Create and delete from the dashboard.** The dashboard lists stores with status (Provisioning / Ready / Failed), URL and created time. Several stores can be provisioned at the same time.
- **Kubernetes-native provisioning.** Each store is provisioned through Helm or the Kubernetes API, in its own namespace, with its own database, Secrets, persistent volumes and Ingress host.
- **Automatic MCP registration.** Each new store gets its own WooCommerce MCP server, registered in the gateway without manual steps. Tool names make it clear which store a tool acts on.
- **Clean teardown.** Deleting a store removes its namespace, volumes, Secrets and its MCP registration.
- **Isolation.** One store's MCP server cannot reach another store's database or WordPress. Show this with NetworkPolicies.
- **Guardrails.** ResourceQuota and LimitRange per store namespace, a maximum number of stores per user, and a provisioning timeout.
- **Recovery.** Creating a store is safe to retry. If the provisioning component restarts mid-provisioning, the store either completes or fails cleanly with a clear reason.

The test for this section: ask the chat agent to compare today's orders across two stores, then delete one store and show that its tools and resources are gone.

Other ways to stand out

These are strong differentiators if implemented well. Pick what interests you; depth in one area beats a thin attempt at all of them.

1) Production-like VPS deployment

- Deploy the same Helm charts on a VPS (k3s) or a cloud free tier and show it working.
- Document what changed through Helm values (domain, ingress, storage, secrets).
- Optional: TLS with cert-manager.

2) Tool permissions

- Per-role or per-user control over which servers and tools are allowed. For example, members can read orders but only admins can create products or change order status.
- The restricted tools are hidden from the tool list and rejected if called directly.

3) OAuth on the gateway

- Replace or complement API keys with the OAuth flow from the MCP authorization specification, so clients can connect with a browser login.

4) Handling many tools

- Ten servers can mean hundreds of tools, which is too many to hand to a model at once. Show an approach, such as tools that search for and then execute other tools.

5) Hosted MCP servers

- Let an admin add a server by container image instead of URL. The platform deploys it into the cluster with its own Secret, limits and NetworkPolicy, and removes it cleanly.

6) Security hardening

- Protection against server-side request forgery when an admin adds a URL (cluster-internal addresses, cloud metadata endpoints).
- Least-privilege RBAC for any component that talks to the Kubernetes API.
- NetworkPolicies that deny by default, and containers that run as non-root.
- Upstream credentials encrypted at rest.

7) Scaling

- Run more than one gateway replica and show that MCP sessions and upstream connections still work.
- Rate limits per user or per key.

8) Observability

- Metrics such as tool calls, error rate and latency per server.
- Clear reporting of why an upstream is unreachable or why a call failed.

9) Upgrades and rollback

- Show or document how you upgrade the store, the MCP server and the gateway with Helm, and how you roll back safely.

Demo video and submission (mandatory)

Submission

Please submit through the form:

- Demo video

- GitHub repo
- Form URL: https://forms.gle/nftwgyNeenSDmiRb6

Demo video requirements

Show the real system you built, covering:

1. **System design and implementation**
    - components and responsibilities
    - the definition-of-done flow, live, with two users
2. **Kubernetes**
    - what runs where: namespaces, Secrets, volumes, Ingress
    - requests and limits, probes
    - what is public and what is internal-only, and how that is enforced
3. **MCP internals**
    - how the gateway connects to upstreams and how a tool call travels from client to store and back
    - how you handle tool name collisions, timeouts and a dead upstream
4. **Security posture**
    - how API keys and upstream credentials are stored and used
    - what a member can and cannot do
    - what happens if an admin adds a malicious or compromised MCP server
5. **Chat agent**
    - the tool loop, and how the agent acts as the logged-in user
6. **Local-to-VPS production story**
    - what changes between local and VPS through Helm values
    - ingress, domains, storage and secrets strategy
    - upgrade and rollback with Helm

Deliverables (in the repo)

- `README.md` including:
    - local setup instructions
    - VPS or production-like setup instructions (k3s is fine)
    - how to place an order, connect an external MCP client, and use the chat agent
- Source code for the dashboard, the backend and gateway, the WooCommerce MCP server and the chat agent
- Helm charts and values files (local and production)
- A short "System design and tradeoffs" note covering:
    - architecture choices
    - how tool aggregation, routing and failure handling work
    - what changes for production (DNS, ingress, storage class, secrets)
- A short threat model note covering:
    - shared upstream credentials: what a member can do with a credential they never see

    - why client tokens are not passed through to upstream servers
    - untrusted upstreams: tool descriptions written to manipulate the model
    - what one team member can do to another
- If you attempted the bonus or any stand-out item, a list of which ones and where to find them